Pluralsight

AI ENGINEERING ACADEMY · WEEK 1

# Agile Fundamentals

Week 1 of 24 - Agile Mindset, Core Workflow & the eComMART AI Transformation Story

# Skills coverage across your entire org

## SOFTWARE DEVELOPMENT

**140+ SKILL ASSESSMENTS | 1600+ COURSES | 450+ LABS | SANDBOXES**

- Frontend web development
- Backend web development
- Mobile development
- Software testing
- Software architecture
- Version control

**Programming languages:**

- Java
- Python
- JavaScript
- C#, C++
- Ruby
- Swift
- Kotlin
- TypeScript
- PHP
- Go

## PRODUCT + BUSINESS MANAGEMENT

**35+ SKILL ASSESSMENTS | 900+ COURSES**

- Business professionals
- Business analysis
- Communication skills
- Leadership skills
- Portfolio, program, and project management
- Productivity apps and client OS
- User experience design
- Product
- Agile
- Interactive experiences

## CLOUD

**90+ SKILL ASSESSMENTS | 1100+ COURSES | 2900+ LABS | SANDBOXES**

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform
- Multi-cloud
- Certifications
- Cloud literacy
- Cloud transformation
- Cloud management and monitoring
- Cloud security
- Serverless workloads
- Big data/ML/AI in the cloud
- Cloud identity access management
- Cloud cost control and optimization
- DevOps in the cloud
- Cloud compute and containerization
- Cloud network management and configuration



**10,000+**

courses, assessments, and hands-on learning experiences from

**2,500+**

expert authors

## GENERATIVE AI

**20 SKILL ASSESSMENTS | 450+ COURSES | 50+ LABS | SANDBOXES**

- Generative AI
- Languages and Libraries
- Chat Bots
- Speech Recognition
- Image Recognition
- Sentence Classification

## SECURITY

**50+ SKILL ASSESSMENTS | 600+ COURSES | 350+ LABS | SANDBOXES**

- Certifications
- Security literacy and fundamentals
- Security architecture and engineering
- Security development
- Security operations
- Security testing
- Governance, compliance, and risk
- Advanced persistent training

## IT OPS

**50+ SKILL ASSESSMENTS | 1300+ COURSES | 1600+ LABS | SANDBOXES**

- IT fundamentals and practices
- Identity management
- Configuration management
- Containers
- DevOps
- Endpoint management
- IT automation
- Management and monitoring
- Messaging and collaboration
- Mobile device management
- Network infrastructure
- Operating systems and stacks
- Server infrastructure
- Storage
- Virtualization
- Container orchestration
- Desktop administration
- Salesforce

## DATA

**90+ SKILL ASSESSMENTS | 700+ COURSES | 450+ LABS | SANDBOXES**

- Data analytics
- Business analytics
- Data engineering
- Data management
- Data science
- Data literacy
- Machine learning
- Managerial skills

*The numbers displayed include all content accessible on our platform and are subject to change.

**Labs and skill assessments may be tagged to multiple domains. This may lead to a display number that is higher than the total sum of labs or skill assessments.

Pluralsight

## About Me:

- Currently, leading GenAI / Agentic AI adoption across leadership and functional teams.
- Learning and reading enthusiast - only long-form content!
- 23+ yrs. exp in knowledge sharing and consulting
- Value Investing practitioner - keeping emotions at bay
- Bachelors in Business Ad, Masters in Computer Applications and Business Ad



# My pledge to you:

## I will…

- Make this course interactive
- Ask you questions
- Ensure everyone can speak
- Create an inclusive learning environment
- Use an on-screen timer for breaks

**…also, if you have an accessibility need, please let me know!**

# Let’s get started!




*Proprietary and confidential*

# Today's Agenda

Everything we cover in this 2-hour session, from the eComMART story to your first backlog.

### The eComMART Story

Meet the squad and the AI idea we are chasing.

### Agile Fundamentals

Mindset, Waterfall vs. iterative, Scrum and Kanban.

### Core Agile Workflow

Epics, stories, SMART, FR vs. NFR, sprints and Definition of Done.

### Agile Transformation

Product teams, cross-functional squads, feedback loops.

### Agile for AI Projects

Why AI features need experiments, not fixed specs.

### Live Demo & Takeaway

Build a real backlog together, then it is your turn.

# Welcome to eComMART

A multinational B2B marketplace that is growing faster than its people can keep up with.

### Buyers wait too long for quotes

RFQs (Requests for Quotation) pile up while suppliers respond by hand.

### Suppliers are exhausted

Manual lead qualification, stock checks and repetitive inquiries.

### You just joined the AI Engineering Squad

Mission: build an Agentic Commerce System over the next 24 weeks.




*Two agents, one mission: Buyer Agent + Seller Agent*

# The Boss Has an Idea...

It is week one at eComMART. Working alone, you might just start coding. Here, we need a plan.

## The Vague Ask

*"I want an AI that qualifies our leads and negotiates for our suppliers." That is it. That is the whole brief from the boss.*

- Tempting to just start coding right away
- But real requirements are never fully known upfront
- We need a plan built for change: Agile

# Why Waterfall Struggled

The "textbook" way: write the plan, build for months, ship it. Software teams hit the same problems for decades.

### Months of Planning

A huge spec is written before any working software exists.

### Built in Isolation

Customers see nothing until the end, so feedback arrives too late.

### Surprises at Launch

Needs have changed, bugs surface late, and fixes cost a lot.

A QUICK HISTORY




***The lesson:*** *Agile is not new - it is decades of proven practice for handling change.*



# Agile Fundamentals

- Evolution an history
- Agile mindset - values, principles & the Agile Manifesto
- Waterfall vs. iterative delivery
- Scrum & Kanban - two ways teams organize their work

# What Is Agile?

Four simple values, adapted from the Agile Manifesto, that anchor everything else this week.

### Individuals & Interactions

People and communication over rigid process.

### Working Software

A usable increment beats a perfect document.

### Customer Collaboration

Buyers and suppliers shape the plan with us.

### Responding to Change

Adjust the plan as we learn more.

***Mindset first:*** *Agile is a way of thinking about uncertainty, not just a set of meetings.*

# The 12 Agile Principles (1 of 2)

The Agile Manifesto is backed by twelve principles. Here are the first six, with what they look like on the squad.

### 1 Early & Continuous Delivery

Deliver value early: a working stock check beats a perfect plan.

### 2 Welcome Change

Late changes are welcome: a new RFQ rule is input, not a threat.

### 3 Deliver Frequently

Ship working software every few weeks, not every few months.

### 4 Business + Devs Together

The Product Owner and engineers talk every single day.

### 5 Motivated Individuals

Give the squad support and trust instead of micromanaging.

### 6 Face-to-Face Conversation

A quick conversation beats a long email chain.

# The 12 Agile Principles (2 of 2)

The remaining six principles that shape how Agile teams work every day.

### 7 Working Software = Progress

Demo real features, not status reports.

### 8 Sustainable Pace

A steady rhythm beats heroic late nights.

### 9 Technical Excellence

Good design and clean code keep future changes cheap.

### 10 Simplicity

Maximize the work not done: build only what is needed.

### 11 Self-Organizing Teams

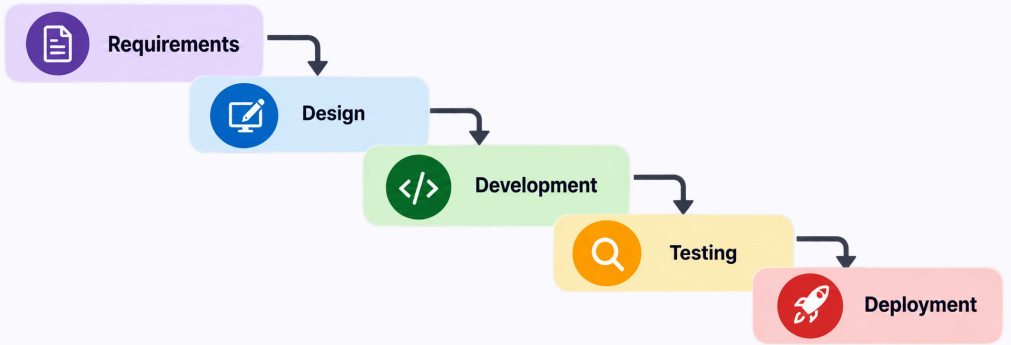
The squad decides how to build; the best designs come from the team.

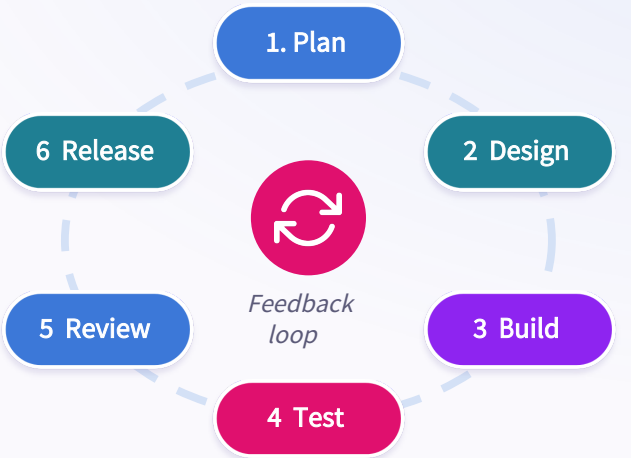
### 12 Reflect & Adjust

Hold regular retrospectives and tune how the team works.

# Waterfall vs. Iterative Delivery

Two very different ways to turn an idea into working software.




## Waterfall

- Plan everything upfront
- Build for months, untested
- One big release, high risk
- Change is expensive

## Iterative (Agile)

- Plan just enough to start
- Build in small, testable slices
- Release often, learn fast
- Change is expected

***The takeaway:*** *Requirements always change - short iterations let the team learn and adapt instead of guessing upfront.*

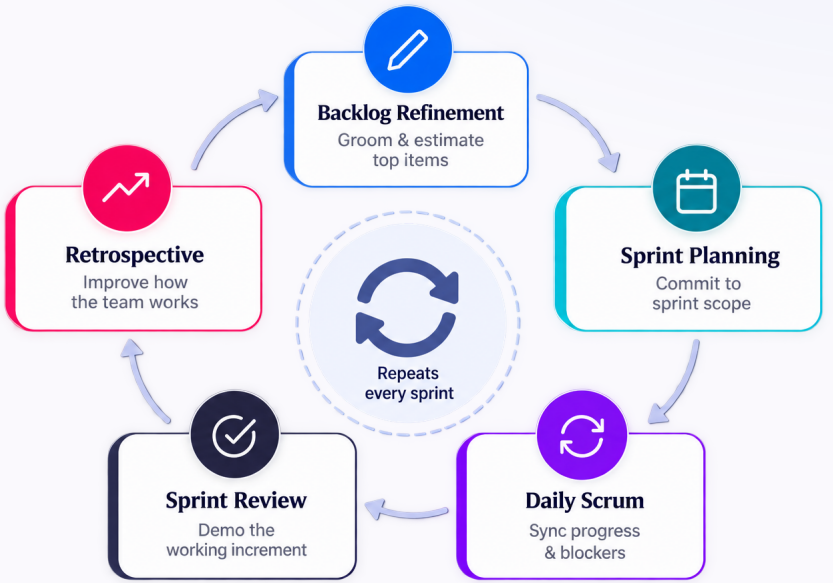
# Cross-Functional Teams: Who Is on the Squad

One team owns a problem end-to-end, not a chain of handoffs.

### Product Owner
Owns the backlog and decides what matters most.

### Scrum Master
Guards the process and clears the squad's blockers.

### Dev + QA + AI Team
Builds, tests, and demos a working slice each sprint.




*Shared ownership:* *the whole squad is accountable for the outcome, not just their own piece.*

# Building the Proof

Everything we build over the next 24 weeks feeds these two agents - built the Agile way.

**What is an MVP?** Minimum Viable Product - the smallest version of the product that delivers real value and lets us learn from real users. For eComMART, the first MVP is a Seller Agent that checks stock and counters offers safely (Sprint 1), not every feature at once.




*Weeks 1-6: we build the "hands" (functions) and "memory" (databases) before any LLM is connected.*

# Epic → User Story → Task: One Worked Example

Follow Epic 1 from big goal, to one story, to the tasks an engineer picks up.

**EPIC**
A big goal - spans many sprints

**USER STORY**
One slice of value - fits in a sprint

**TASKS**
Work an engineer does - hours to days

## Epic 1
### Seller Agent Checks Inventory

Suppliers never quote items they do not have. Contains Story 1.1 and Story 1.2.

## Story 1.1
### Check stock before quoting

*"As a supplier, I want the Seller Agent to check my stock before quoting, so that I never promise items I do not have."*

- **1.1.a FR:** Write check_stock(sku)
- **1.1.b FR:** ProductCatalog loads JSON
- **1.1.c NFR:** Reply under 5 s in 95% of 20 runs
- **1.1.d NFR:** No invented stock, 20 of 20 cases

***Remember:*** *every task traces back to a story, and every story traces back to an epic.*

# Applying the SMART Criteria

Use SMART to make every User Story and Acceptance Criterion verifiable.

### S

### Specific

One clear outcome, no ambiguity

### M

### Measurable

A number or pass/fail you can check

### A

### Achievable

Doable by the squad in one sprint

### R

### Relevant

Tied to the epic and business value

### T

### Time-bound

Done by a set sprint or date

***Rule of thumb:*** *if you cannot test a story, it is not SMART yet.*

# SMART in Action: Before & After

The same requirement, before and after applying SMART (Story 1.1).

## Vague (not SMART)

- "The AI should check stock quickly."
- No target, no scope, no deadline
- Impossible to test
- Every engineer may interpret it differently

## SMART

**Story:** "As a supplier, I want the Seller Agent to check my stock before quoting, so that I never promise items I do not have."

**S Specific:** reads stock from the product catalog (ProductCatalog JSON) for every quote

**M Measurable:** reply under 5 s in 95% of 20 test runs; no invented stock in 20 of 20 cases

**A Achievable:** one function, check_stock(sku), buildable in one sprint

**R Relevant:** supports Epic 1 - suppliers never quote items they do not have

**T Time-bound:** demoed at the Sprint 1 Review

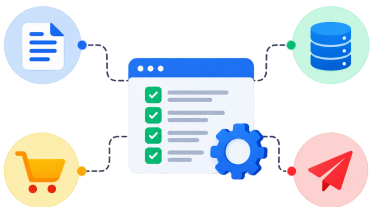
***Notice:*** *the SMART version can be tested by anyone on the squad.*

# Functional vs. Non-Functional Requirements

Requirements describe both what the system does and how well it must do it.

## Functional (what it does)

- Behaviors and features
- "Fetch a lead's details"
- "Check inventory for a SKU"
- "Send a price counter-offer"



## Non-Functional (how well)

- Quality and performance limits
- Response latency under 5 seconds
- Hallucination rate below 2% on test set
- Security, reliability, auditability



*Why it matters:* *a feature that works but hallucinates is not "done".*

# Sorting eComMART Requirements

For each requirement, is it functional or non-functional?

| | |
|---|---|
| Seller Agent checks live inventory before replying | Functional |
| Quote replies arrive within 5 seconds | Non-Functional |
| Buyer Agent submits an RFQ to a supplier | Functional |
| Fabricated prices in replies stay below 2% | Non-Functional |
| Lead is scored using a standard business framework | Functional |
| Every negotiation decision is logged for audit | Non-Functional |

# Product Backlog vs. Sprint Backlog

Two lists, two very different jobs - shown with the eComMART stories.

## Product Backlog

- Every idea for the whole product
- Owned and prioritized by the Product Owner
- Lives for the life of the product



**Story 1.2** Answer stock inquiries (Sprint 2)

**Story 2.1** Score leads with BANT (Sprint 2)

**Story 2.3** Flag unusual requests (Sprint 2)

**Story 3.1** Submit an RFQ (Sprint 3)

**Story 3.2** Check RFQ status (Sprint 3)

## Sprint Backlog

- Only this sprint's committed work
- Owned by the development team
- Reset at every sprint planning



**Story 1.1** Check stock before quoting - 4 tasks

**Story 2.2** Counter low offers automatically - 5 tasks

*Sprint 1 total: 2 stories, 9 tasks (4 FR + 5 NFR)*

*Sprint = a fixed timebox (usually 1-4 weeks) that ends with a potentially shippable increment.*

# Sprint Planning, Review & Retrospective

The three events that open, showcase, and improve every sprint.

## Sprint Planning

Team picks backlog items, sets a Sprint Goal, and breaks stories into tasks.

## Sprint Review

Demo the working increment to stakeholders and gather feedback.

## Sprint Retrospective

Inspect how the team worked and pick one improvement for next sprint.

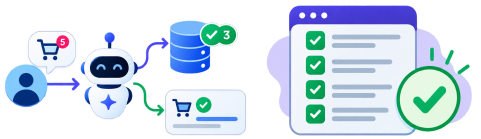
***Rhythm:*** *Plan → Build → Review → Retro, then repeat with what you learned.*

# Acceptance Criteria vs. Definition of Done

Two different checklists - one says what must be built, the other says whether it is truly done.

## Acceptance Criteria: what was to be done

- Given a supplier has 3 units in stock
- When a buyer requests 5 units
- Then the agent replies with the true availability
- Written separately for each user story



## Definition of Done: done or not done?

- Code reviewed and merged
- Tests written and passing
- Demoed to the Product Owner
- Same checklist for every story, every sprint



*The difference:* *Acceptance Criteria show what was to be done. Definition of Done answers: done or not done?*

# Scrum Board: Sprint 1

The whole sprint on one board - the eComMART squad's plan for Sprint 1.

**Sprint Goal: Seller Agent checks stock and counters offers safely**

**Story 1.1** Check stock before quoting

**Story 2.2** Counter low offers automatically

| Product Backlog | Sprint 1 To Do | In Progress | Review / Testing | Done |
|---|---|---|---|---|
| **Story 1.2** Answer stock inquiries (Sprint 2) | **1.1.c NFR:** Reply under 5 s in 95% of 20 runs | **1.1.a FR:** Write check_stock(sku) | **2.2.b FR:** Pass stock count into counter_offer | **Epic approved and sized** |
| **Story 2.1** Score leads with BANT (Sprint 2) | **1.1.d NFR:** No invented stock, 20 of 20 cases | **2.2.a FR:** Write counter_offer(listed, offer, floor, stock) | | **1.1.b FR:** ProductCatalog loads JSON |
| **Story 2.3** Flag unusual requests (Sprint 2) | **2.2.c NFR:** Counter never below floor price | | | |
| **Story 3.1** Submit an RFQ (Sprint 3) | **2.2.d NFR:** No invented prices | | | |
| **Story 3.2** Check RFQ status (Sprint 3) | **2.2.e NFR:** Counter reply under 5 s | | | |

Yellow = User Story

Blue = Functional task (FR)

Pink = Non-functional task (NFR)

Green = Done item

# Scrum Board by Story: Story 1.1

One user story, split into the tasks the squad moves across the board.

**Story 1.1**
Check stock before quoting

*"As a supplier, I want the Seller Agent to check my stock before quoting, so that I never promise items I do not have."*

**Epic 1: Seller Agent Checks Inventory**
1 of 4 tasks done

| To Do | In Progress | Review / Testing | Done |
|---|---|---|---|
| **1.1.c NFR:** Reply under 5 s in 95% of 20 runs | **1.1.a FR:** Write check_stock(sku) | | **1.1.b FR:** ProductCatalog loads JSON |
| **1.1.d NFR:** No invented stock, 20 of 20 cases | | | |

Yellow = User Story

Blue = Functional task (FR)

Pink = Non-functional task (NFR)

Green = Done item

# Product-Oriented vs. Project-Oriented Delivery

A mindset shift that changes how teams are structured and measured.

## Project-Oriented

- Temporary team, then disbands
- Success = on time, on budget
- Handed off once "done"
- Scope fixed upfront



## Product-Oriented

- Long-lived team owns the outcome
- Success = value delivered to users, fast
- Keeps improving after launch
- Scope evolves with feedback



*eComMART squad is product-oriented: it owns the Buyer & Seller Agents for their entire life.*

# Prototype vs. Production-Ready AI

Knowing which one you are building changes how much rigor the work needs right now.

## Prototype

- Proves the idea can work at all
- Hand-checked, small sample of cases
- Okay to be rough around the edges
- Goal: learn fast

## Production-Ready

- Handles messy, real-world input
- Tested, monitored, and logged
- Safe to fail gracefully, every time
- Goal: trust at scale

***Week 1 rule of thumb:*** *every AI feature starts as a prototype - never skip straight to production.*

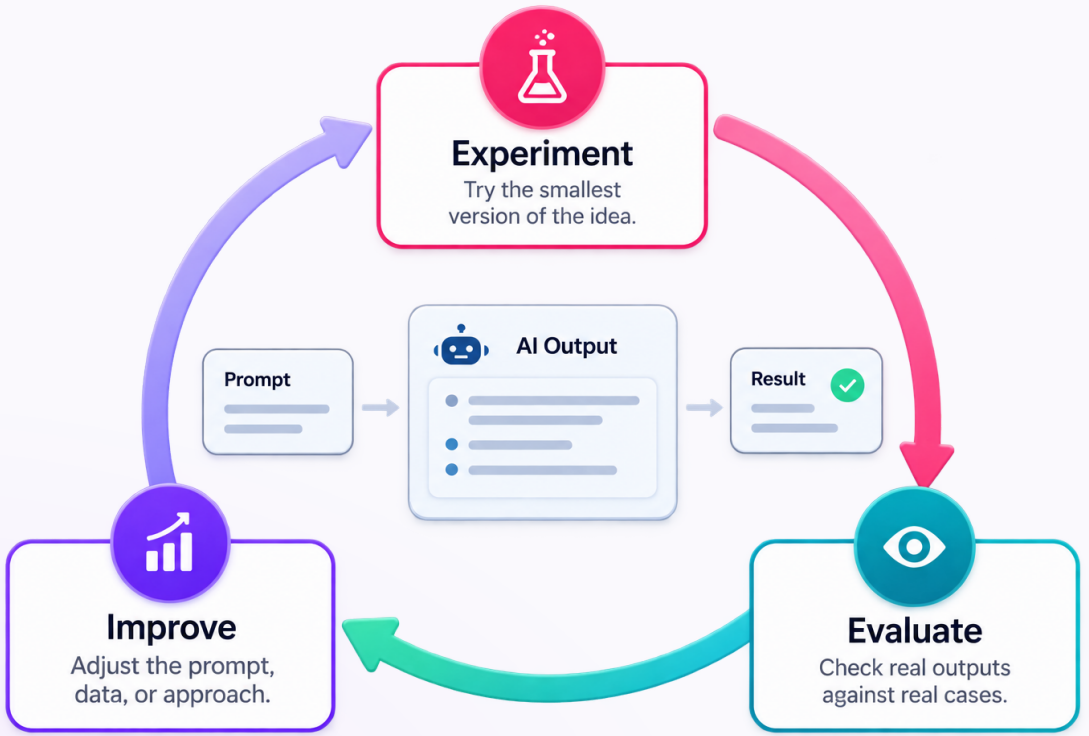


# Agile for AI Projects

- Experiment → Evaluate → Improve loop
- Handling uncertain requirements and outputs
- Prototype vs. production-ready solution

# The AI Twist: Experiment → Evaluate → Improve

AI output is never fully predictable, so the loop itself becomes the plan.






# Handling Uncertain Requirements & Outputs

You cannot fully specify AI behavior upfront, so design short experiments you can verify.

### Define a Hypothesis

"If the agent uses inventory data, hallucinated stock replies drop below 2%."

### Measure with Test Cases

Use a fixed set of real examples so results are comparable.

### Human Validation

A person reviews outputs before we trust them.

### Capture Failures

Every bad output becomes a new test case.

*Remember:* *a requirement for AI should always include how we will evaluate it.*

# From Vague Idea to Agile Backlog

The instructor plays Product Manager. Together we turn the boss's idea into a real backlog.

## Let's Build It Together

*"I want an AI that qualifies our leads and negotiates for our suppliers." We are turning this into eComMART's first Agile backlog, live, as a class.*

- Step 1: Write the Epics
- Step 2: Break them into User Stories
- Step 3: Define Acceptance Criteria
- Step 4: Build the MVP Backlog

# Writing the Epics

Every backlog starts with clear, ambitious goals the whole team can rally around.

## The Boss's Idea, Clarified

*As eComMART, we want AI agents that check stock, qualify leads, negotiate offers and speed up RFQs, so suppliers spend less time on repetitive work and buyers wait less.*

### Epic 1: Seller Agent Checks Inventory

Suppliers never quote items they do not have.

### Epic 2: Lead Qualification and Negotiation

Qualify leads and counter offers safely.

### Epic 3: Buyer Agent Speeds Up RFQs

Buyers submit and track RFQs faster.

# Breaking Epics into User Stories

Each epic splits into small, independently testable slices.

## Epic 1: Seller Agent Checks Inventory

**Story 1.1 Check stock before quoting**
*As a supplier, I want the agent to check my stock before quoting, so I never promise what I do not have.*

**Story 1.2 Answer stock inquiries**
*As a buyer, I want instant answers on stock, so I can plan my purchase.*

## Epic 2: Lead Qualification and Negotiation

**Story 2.1 Score leads with BANT**
*As a supplier, I want leads scored on Budget, Authority, Need and Timeline, so I focus on serious buyers.*

**Story 2.2 Counter low offers automatically**
*As a supplier, I want low offers countered automatically, so I save negotiation time.*

**Story 2.3 Flag unusual requests**
*As a supplier, I want unusual requests flagged to me, so I stay in control.*

## Epic 3: Buyer Agent Speeds Up RFQs

**Story 3.1 Submit an RFQ**
*As a buyer, I want to submit an RFQ in one step, so I get quotes faster.*

**Story 3.2 Check RFQ status**
*As a buyer, I want to check my RFQ status, so I do not have to chase suppliers.*

*NFRs for Sprint 1: replies under 5 s (95% of 20 runs) · no invented stock or prices · never counter below the floor price.*

# Defining Acceptance Criteria

A concrete Given / When / Then scenario for Story 2.2 - what was to be done, in a form we can test.

**Given** A supplier lists an item at $100 with a floor price of $80 and 3 units in stock.

**When** A buyer offers $70 for the item.

**Then** The agent checks stock and counters at $85 - never below the $80 floor price.

***NFR check:*** *the counter arrives within 5 seconds, and no price is ever invented.*

# The MVP Backlog

Prioritized, sized and sequenced - Sprint 1 is our MVP.

## MVP · Sprint 1 (committed)

*Goal: Seller Agent checks stock and counters offers safely*

- **Story 1.1** Check stock before quoting — Must · 5 pts
- **Story 2.2** Counter low offers automatically — Must · 8 pts

13 points · 9 tasks

## Next · Sprint 2

*Goal: answer inquiries, score leads, flag the unusual*

- **Story 1.2** Answer stock inquiries — Should · 3 pts
- **Story 2.1** Score leads with BANT — Should · 5 pts
- **Story 2.3** Flag unusual requests — Should · 3 pts

11 points

## Later · Sprint 3

*Goal: the Buyer Agent speeds up RFQs*

- **Story 3.1** Submit an RFQ — Could · 5 pts
- **Story 3.2** Check RFQ status — Could · 3 pts

8 points

*MVP = only what Sprint 1 commits to. Everything else waits until the squad has learned from it.*

# Practice on Your Own

Self-paced, in your own time - we will build on this in Week 2.

## Your Takeaway Exercise

*Create 5 user stories with acceptance criteria for an AI-enabled business application of your choice - self-paced, before next week.*

Scan to view Kanban & Scrum boards

Deliverable: 5 stories + acceptance criteria

Check each with INVEST and SMART, include at least 1 NFR

Due: before next week's live session

# Key Takeaways: Week 1

Four ideas that will anchor every week that follows.



### Agile Beats a Fixed Plan

Especially when AI behavior is uncertain.

### Epic → Story → Task

The chain that turns a vague idea into real work.

### SMART + FR / NFR

Make stories testable and capture quality limits.

### AC & DoD

Two checklists for "ready" and "done".

*Experiment → Evaluate → Improve is the extra loop AI features need on top of Scrum.*



Pluralsight

# Thank You

See You Next Week: Serving in Agile Teams & Collaboration



# Scrum Board by Story: Story 2.2

Same sprint, second story - the tasks behind countering low offers safely.

**Story 2.2**
Counter low offers automatically

*"As a supplier, I want the Seller Agent to counter low offers automatically, so that I save negotiation time."*

**Epic 2: Lead Qualification and Negotiation**
0 of 5 tasks done

| To Do | In Progress | Review / Testing | Done |
|---|---|---|---|
| **2.2.c NFR:** Counter never below floor price | **2.2.a FR:** Write counter_offer(listed, offer, floor, stock) | **2.2.b FR:** Pass stock count into counter_offer | |
| **2.2.d NFR:** No invented prices | | | |
| **2.2.e NFR:** Counter reply under 5 s | | | |

Yellow = User Story

Blue = Functional task (FR)

Pink = Non-functional task (NFR)

Green = Done item

# What is Kanban?

A visual board where work flows left to right - with a cap on how much is in progress.

### Visualize the work

Every task is a card on a shared board anyone can read.

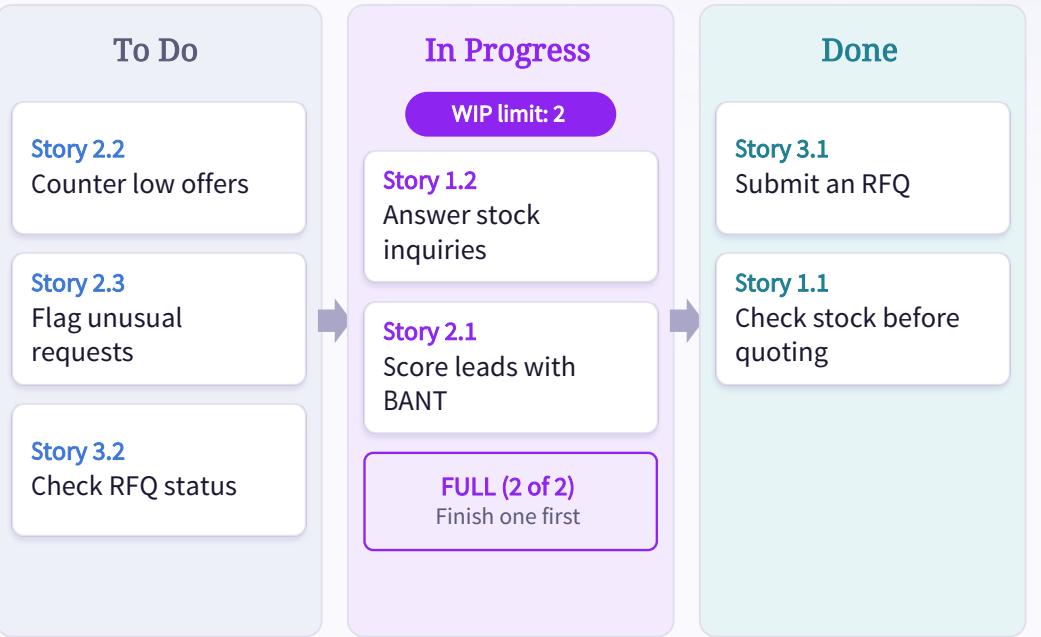
### Limit work in progress

WIP

Cap what is in flight so work gets finished, not just started.

### Manage the flow

Pull new work in only when a slot frees up. No sprints.




*Kanban = a visual board plus a WIP limit: new work is pulled in only when there is capacity.*

# Kanban Board: Work-in-Progress Limits

No sprints. Work flows left to right, one lane per epic, and work in progress is limited.

| | Backlog | To Do | In Progress (WIP limit 2) | Review | Done |
|---|---|---|---|---|---|
| | | | FULL (2 of 2): finish first | | |
| **Epic 1: Seller Agent Checks Inventory** | | **Story 1.2** Answer stock inquiries | **Story 1.1** Check stock before quoting | | |
| **Epic 2: Lead Qualification and Negotiation** | **Story 2.3** Flag unusual requests | **Story 2.1** Score leads with BANT | **Story 2.2** Counter low offers | | |
| **Epic 3: Buyer Agent Speeds Up RFQs** | **Story 3.1** Submit an RFQ<br>**Story 3.2** Check RFQ status | | | | **MVP scope agreed** |

*Key idea:* *finish what is in progress before pulling new work in.*

# Kanban in Motion: Pull, Do Not Push

The same board a day later - finished work frees up a slot, and the next item is pulled in.

| | Backlog | To Do | In Progress (WIP limit 2) | Review | Done |
|---|---|---|---|---|---|
| | | | FULL (2 of 2): finish first | | |
| **Epic 1: Seller Agent Checks Inventory** | | | **Story 1.2** Answer stock inquiries | **Story 1.1** Check stock before quoting | |
| **Epic 2: Lead Qualification and Negotiation** | **Story 2.3** Flag unusual requests | **Story 2.1** Score leads with BANT | **Story 2.2** Counter low offers | | |
| **Epic 3: Buyer Agent Speeds Up RFQs** | **Story 3.1** Submit an RFQ<br>**Story 3.2** Check RFQ status | | | | **MVP scope agreed** |

***What changed:*** *Story 1.1 moved to Review, so Story 1.2 was pulled into In Progress - WIP stays at 2.*

# Scrum vs. Kanban: When to Use Which

Two ways to organize work - the right choice depends on how predictable the work is.

## Scrum

- Fixed-length sprints
- Plan, build, review, retro
- Roles: PO, SM, Dev Team
- Great for coordinated releases

## Kanban

- Continuous flow, no sprints
- Limit work-in-progress
- Visual board, any time changes
- Great for support & ongoing work

***Most real teams blend both:*** *Scrum's rhythm for planning, Kanban's visibility for daily execution.*